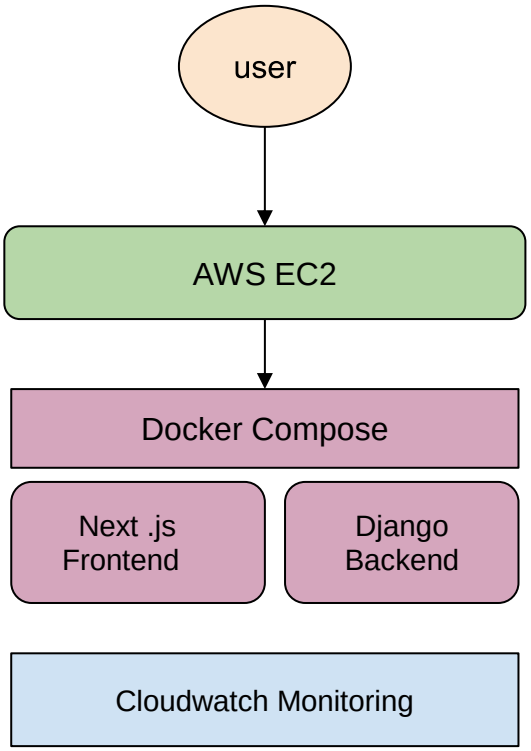
DevOps Intern Assignment- Documentation

Objective: Deploy and monitor two codebases: a Next.js application and a Django Python application. Configure them to run seamlessly and include tasks to test problem-solving abilities**.**

1. Final Architecture Overview



Infrastructure Provisioned by Terraform Code

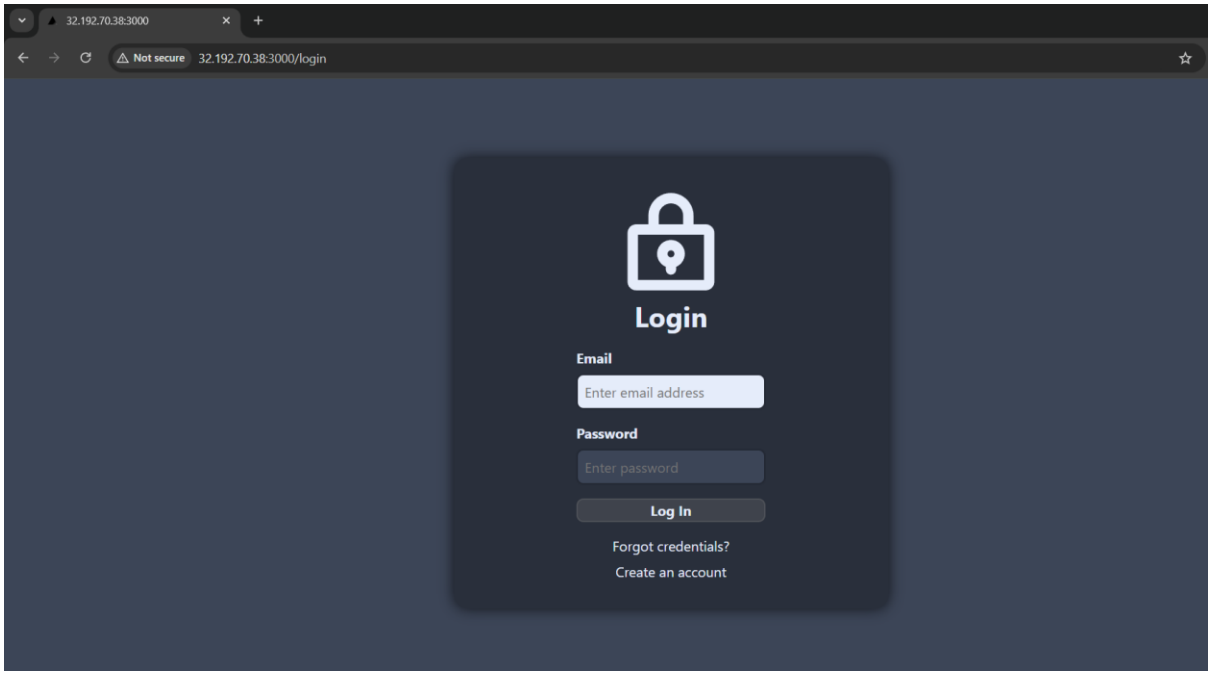
Scope Includes:

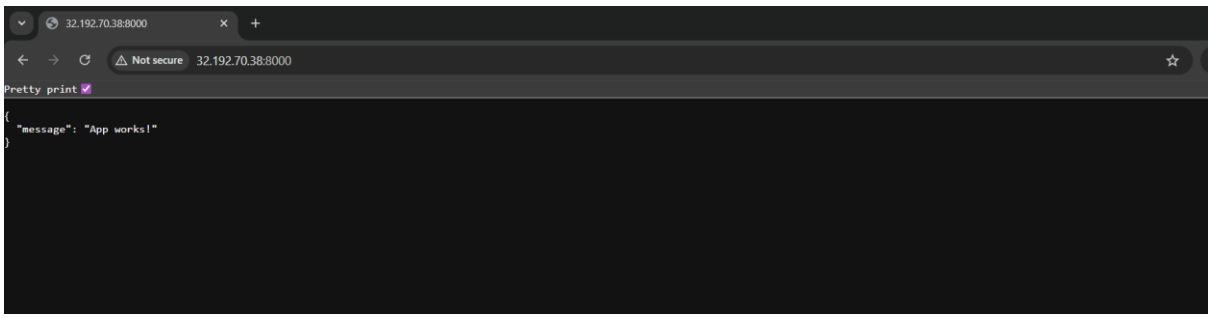
- ➔ EC2 Manual Deployment
- ➔ CI/CD using GitHub Actions
- ➔ Docker Containerization
- ➔ Terraform Infrastructure as Code
- ➔ CloudWatch Monitoring

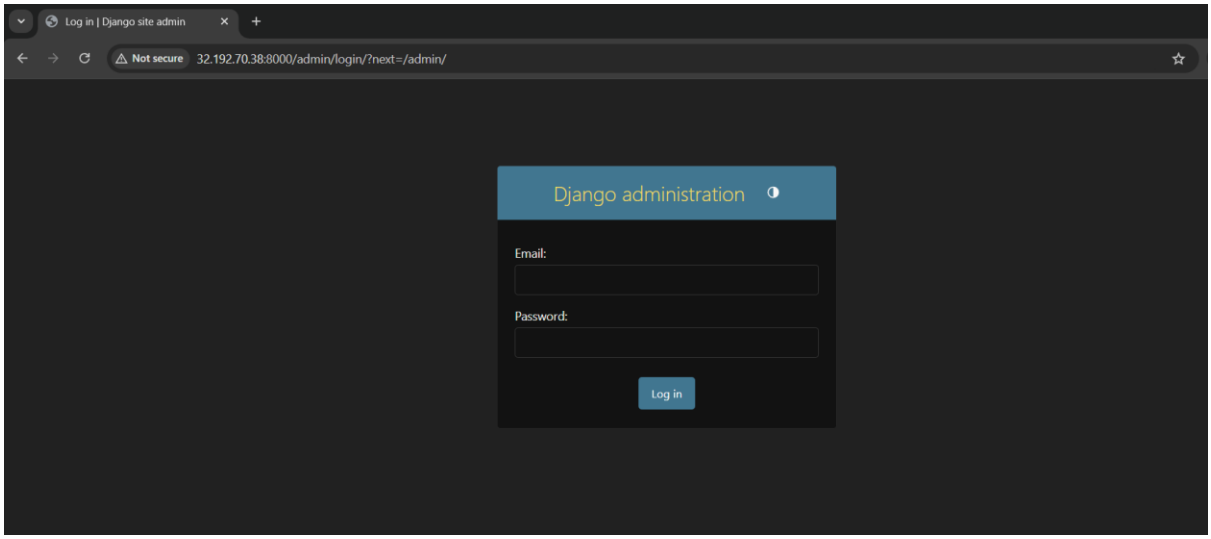
2. Manual EC2 Deployment

1. Launch Ubuntu EC2 instance.
2. Configure Security Group (22, 8000, 3000).
3. SSH into EC2.
4. Install Python, Node.js, Git.
5. Create a virtual environment.
6. Install backend dependencies.
7. Configure .env variables.
8. Run Django server on port 8000.
9. Build and run Next.js on port 3000.

Output screenshots :

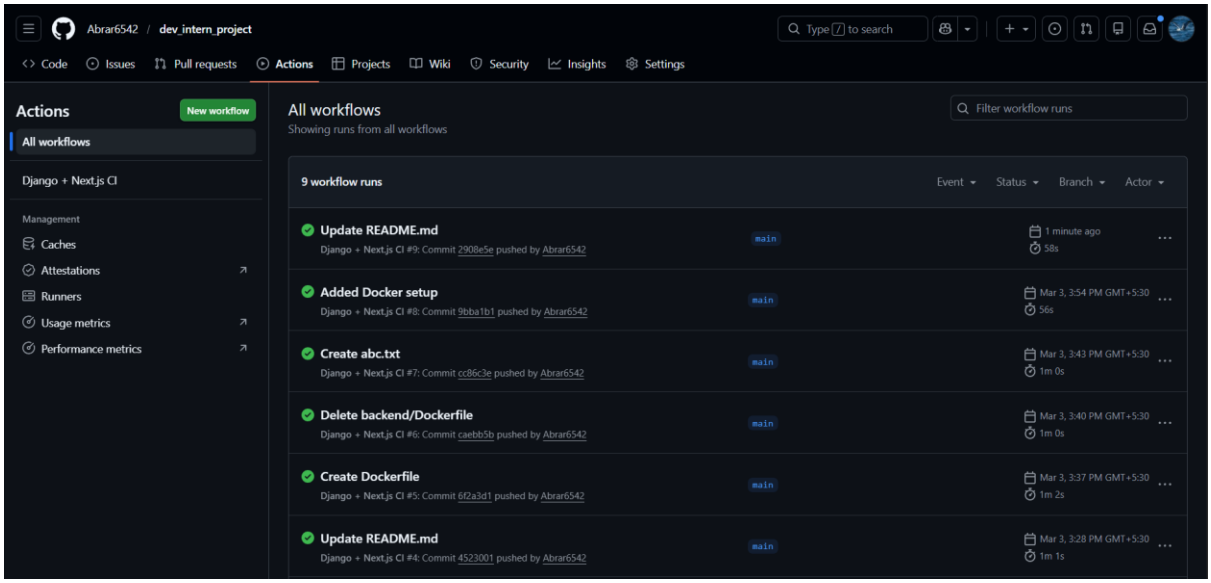






3. CI/CD Pipeline Implementation

1. Create a GitHub repository.
2. Push project source code.
3. Create .github/workflows/ci.yml.
4. Configure steps:
    - Checkout repository
    - Setup Python 3.11
    - Install backend dependencies
    - Run Django system checks
    - Setup Node 18
    - Install frontend dependencies
    - Build frontend
5. Verify green check in GitHub Actions.



4. Docker Containerization

1. Create backend Dockerfile.
2. Create frontend Dockerfile.
3. Create docker-compose.yml.
4. Define services and environment variables.
5. Build containers: docker compose up --build
6. Verify running containers using docker ps.

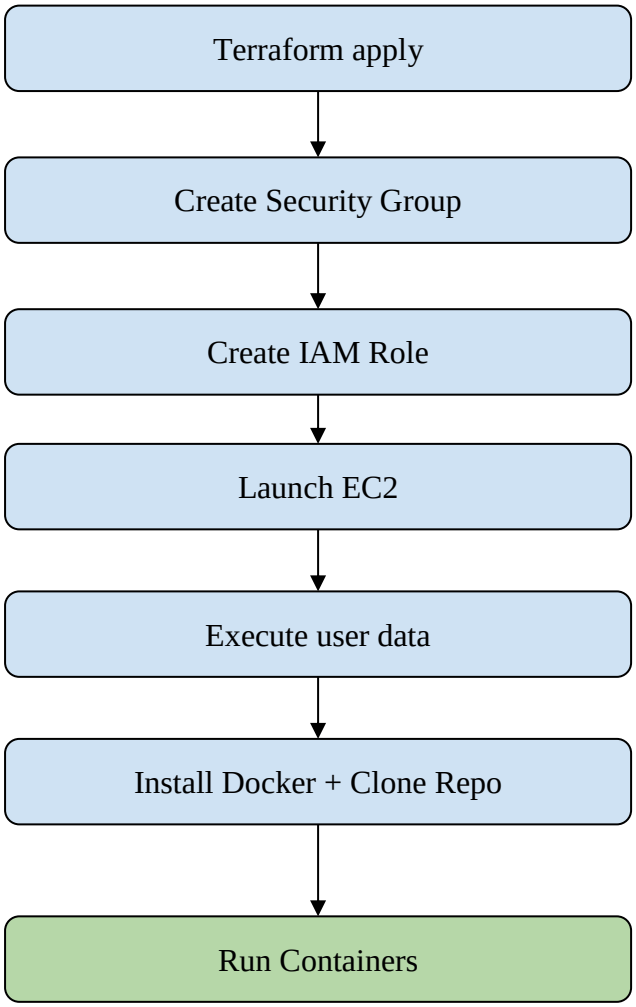
```
ubuntu@ip-10-0-5-137:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-10-0-5-137:~$ docker image ls
REPOSITORY                    TAG         IMAGE ID       CREATED         SIZE
dev_intern_project_frontend   latest      f0bfb18fa7d5   20 hours ago    1.68GB
dev_intern_project_backend    latest      24d39ed608cf   20 hours ago    260MB
python                        3.11-slim   992921a8b23a   7 days ago      124MB
node                          18          b50082bc3670   11 months ago   1.09GB
```

```
ubuntu@ip-10-0-5-137:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-10-0-5-137:~$ docker image ls
REPOSITORY                    TAG         IMAGE ID       CREATED         SIZE
dev_intern_project_frontend   latest      f0bfb18fa7d5   20 hours ago    1.68GB
dev_intern_project_backend    latest      24d39ed608cf   20 hours ago    260MB
python                        3.11-slim   992921a8b23a   7 days ago      124MB
node                          18          b50082bc3670   11 months ago   1.09GB
ubuntu@ip-10-0-5-137:~$ docker ps -a
CONTAINER ID   IMAGE                         COMMAND                  CREATED        STATUS                     PORTS     NAMES
f94c8a96cc74   dev_intern_project_frontend   "docker-entrypoint.s…"   20 hours ago   Exited (1) 19 hours ago            nextjs_frontend
c8c7ba514828   dev_intern_project_backend    "python manage.py ru…"   20 hours ago   Exited (0) 19 hours ago            django_backend
ubuntu@ip-10-0-5-137:~$
```

5. Terraform Infrastructure Automation

1. Create provider.tf (AWS region).
2. Create variables.tf.
3. Create main.tf:
4. Create outputs.tf.
5. Run terraform init.
6. Run terraform apply.
7. EC2 auto installs Docker.
8. Repository auto cloned.
9. docker compose auto executed.

Terraform Deployment Flow Diagram




```
ubuntu@ip-10-0-5-137:~/terraform_project$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.34.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
aws   Search [Alt+S]   United States (N. Virginia)   Ali Baig (2300-3581-8843)   Ali Baig

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  - public_ip = "32.192.71.118" -> (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.dev_server: Creating...
aws_instance.dev_server: Still creating... [00m10s elapsed]
aws_instance.dev_server: Creation complete after 12s [id=i-0b96f90ec95cb37d2]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

public_ip = "44.220.140.173"
```

6. Monitoring with CloudWatch

1. Install CloudWatch Agent on EC2.
2. Configure memory and disk metrics.
3. Create IAM Role with CloudWatchAgentServerPolicy.
4. Attach role to EC2.
5. View logs in CloudWatch Log Groups.

7. Technologies Used

Cloud Server **:** AWS EC2
Monitoring **:** AWS Cloudwatch
Iac tool **:** Terraform
Containerisation tool **:** Docker & Docker Compose
CICD tool : GitHub Actions
Backend : Django
Frontend : Next.js
OS (AMI) : Ubuntu Linux

8. Final Outcome

The project demonstrates:

- Automated infrastructure provisioning
- Containerized architecture
- Continuous Integration pipeline
- Infrastructure as Code implementation
- Centralized monitoring
- Complete DevOps lifecycle

Github Link : https://github.com/Abrar6542/dev_intern_project.git

Submitted by : Abrar Ali Baig